Internet of Things (IoT)

Unit 4: Sensors and Actuators Interfacing with Arduino Uno

Complete, Easy-to-Understand, Exam-Ready Study Notes

Based on the complete provided Unit-4 presentation

Document	Complete Study Notes
Subject	Internet of Things (IoT)
Unit	Unit 4 — Sensors and Actuators Interfacing with Arduino Uno
Purpose	Single self-contained exam preparation resource
Coverage	All slides, definitions, working principles, interfacing, code, formulas, applications, revision and practice questions

Table of Contents

Chapter 1: Introduction to Sensors and Actuators 4

1.1 Why sensors and actuators are important . . . 4

1.2 Sensor vs actuator . . . 4

1.3 IoT system structure . . . 4

1.4 Sensor definition . . . 4

1.5 Sensor classifications . . . 4

1.6 Sensors and actuators covered . . . 4

Chapter 2: MQ-02/MQ-05 Gas Sensor 5

2.1 Definition and purpose . . . 5

2.2 MQ series sensors . . . 5

2.3 Pins and interfacing . . . 5

2.4 Working principle . . . 5

2.5 Arduino code . . . 6

2.6 Applications and exam points . . . 6

Chapter 3: Obstacle Sensor 7

3.1 Definition . . . 7

3.2 Pins and interfacing . . . 7

3.3 Working principle . . . 7

3.4 Arduino code . . . 7

3.5 Applications and exam points . . . 7

Chapter 4: HC-SR04 Ultrasonic Distance Sensor 8

4.1 Definition . . . 8

4.2 Construction and pins . . . 8

4.3 Features . . . 8

4.4 Working principle . . . 8

4.5 Timing diagram . . . 9

4.6 Arduino functions . . . 9

4.7 Distance formula . . . 9

4.8 Arduino code . . . 9

4.9 Applications and exam points . . . 9

Chapter 5: LDR Light Sensor 10

5.1 Definition . . . 10

5.2 Working . . . 10

5.3 Voltage divider and interfacing . . . 10

5.4 Arduino code . . . 10

5.5 Applications and exam points . . . 10

Chapter 6: Heartbeat Sensor 11

6.1 Definition . . . 11

6.2 Pins . . . 11
6.3 Working . . . 11
6.4 BPM formula . . . 11
6.5 Code logic . . . 11
6.6 Applications and exam points . . . 12
Chapter 7: Actuators **13**
7.1 Definition . . . 13
7.2 Working . . . 13
7.3 Types . . . 13
7.4 Sensor vs actuator . . . 13
8. Relay . . . 13
8.1 Terminals . . . 13
8.2 Working . . . 13
8.3 Connection and code . . . 14
8.4 Applications . . . 14
Chapter 9: Servo Motor **15**
9.1 Definition . . . 15
9.2 Working . . . 15
9.3 Servo library . . . 15
9.4 Code examples . . . 15
9.5 Arduino code . . . 15
9.6 Applications and exam points . . . 16
Chapter 10: Complete Final Revision **17**
10.1 Summary . . . 17
10.2 Definitions . . . 17
10.3 Formulas/syntax . . . 17
10.4 Comparisons . . . 18
10.5 Checklist . . . 18
Chapter 11: Exam Practice Questions **19**
11.1 Short answers . . . 19
11.2 Long answers . . . 19
11.3 MCQs . . . 19
Final One-Page Revision **20**
Core rule . . . 20
Key facts . . . 20
Functions . . . 20
Answer rule . . . 20

Chapter 1: Introduction to Sensors and Actuators

1.1 Why sensors and actuators are important

Sensors and actuators are critical components of embedded systems. An embedded system is a small computer system built inside a larger device to perform a specific task. They are used in aircraft flight-control systems, nuclear-reactor process-control systems, and power plants requiring automated control.

A sensor helps a device understand its surroundings; an actuator helps it perform an action. Example: a temperature sensor measures room temperature and a motor/fan actuator turns the fan ON or changes speed.

1.2 Sensor vs actuator

Point	Sensor	Actuator
Main purpose	Tracks environmental changes by measurement	Controls physical changes by performing action
Role	Input device	Output device
Direction	Environment → Arduino	Arduino → Real world
Example	LDR, gas sensor, ultrasonic sensor	Motor, relay, LED, servo motor
Question answered	"What is happening?"	"What action should be done?"

Memory: A sensor senses the environment; an actuator acts on the environment.

1.3 IoT system structure

An IoT device is made up of: **Physical object ("thing") + Controller ("brain") + Sensors + Actuators + Network/Internet.**

The controller, usually Arduino Uno, receives sensor data, processes it, and sends control signals to actuators. The network allows data exchange over the Internet.

Flow: Environment → Sensor → Arduino/controller → Actuator → Physical action.

1.4 Sensor definition

Definition: A sensor is a device that detects changes and events in a physical environment and converts physical parameters into electrical signals.

It can measure humidity, pressure, temperature, heat, motion, light, gas, distance, and heartbeat. The electrical signal can be displayed or sent over a network for processing.

Example: An LDR detects light; its resistance changes with light, and Arduino reads this change.

1.5 Sensor classifications

Based on power supply:

Type	Meaning	Example
Active sensor	Requires external power supply	Ultrasonic sensor, IR emitter-based sensor
Passive sensor	Does not require external power supply	LDR and similar passive sensors

Based on output signal:

Type	Output	Arduino function
Analog sensor	Continuous/variable voltage or value	analogRead()
Digital sensor	Only HIGH/LOW or 1/0	digitalRead()

Sensors are input devices. Analog sensors generate analog output voltage; digital sensors generate digital output voltage.

1.6 Sensors and actuators covered

This unit covers MQ-02/MQ-05 gas sensor, obstacle sensor, HC-SR04 ultrasonic sensor, LDR sensor, heartbeat sensor, relay, and servo motor. Soil-moisture, temperature-humidity, and flame sensors are also listed as common examples.

Common mistake: Active/passive is based on power supply; analog/digital is based on output signal type.

Chapter 2: MQ-02/MQ-05 Gas Sensor

2.1 Definition and purpose

Definition: The MQ-02/MQ-05 is a gas sensor used to detect combustible gases and smoke.

MQ-02 detects smoke, hydrogen, LPG, methane, and alcohol. MQ-05 detects natural gas and LPG but is not sensitive to smoke.

2.2 MQ series sensors

Sensor	Gases detected
MQ-2	Methane, butane, LPG, smoke
MQ-3	Alcohol, ethanol, smoke
MQ-4	Methane, CNG gas
MQ-5	Natural gas, LPG
MQ-6	LPG, butane gas
MQ-7	Carbon monoxide
MQ-8	Hydrogen gas
MQ-9	Carbon monoxide, flammable gases
MQ-131	Ozone
MQ-135	Air quality: CO, ammonia, benzene, alcohol, smoke
MQ-136	Hydrogen sulfide gas
MQ-137	Ammonia
MQ-138	Benzene, toluene, alcohol, acetone, propane, formaldehyde, hydrogen
MQ-214	Methane, natural gas

2.3 Pins and interfacing

Pin	Connection/purpose
VCC	Arduino 5V
GND	Arduino GND
DO	Digital output pin
AO	Analog output pin

For analog measurement, connect AO to Arduino A0 and read with analogRead(A0).

Memory: VCC power, GND circuit, AO analog, DO digital.

2.4 Working principle

1. Flammable gas/smoke passes through sensing coil.
2. Coil reacts; internal resistance decreases.
3. Voltage across sensor changes/increases.
4. Variable voltage appears at AO.
5. Arduino reads using analogRead(A0).

If gas present, voltage high; higher reading means higher gas/smoke level.

2.5 Arduino code

```
int gas_level; void setup() { pinMode(A0, INPUT); Serial.begin(9600); } void loop() { gas_level =
analogRead(A0); Serial.print("Gas Level : "); Serial.println(gas_level); delay(500); }
```

Code line	Explanation
int gas_level;	Stores gas reading.
pinMode(A0, INPUT);	Sets A0 input.
Serial.begin(9600);	Starts serial communication.
analogRead(A0);	Reads analog voltage.
Serial.print/println	Displays text/value.
delay(500);	Waits 500 ms.

Output: Serial Monitor shows “Gas Level :” and numeric value; value rises with gas/smoke.

2.6 Applications and exam points

- Gas-leak detection, smoke/fire alarms, LPG safety, air-quality monitoring.
- Low cost, simple interfacing, analog+digital outputs.
- MQ-2 detects smoke/LPG/methane/alcohol; gas reduces resistance.

Common confusion: MQ-2 is smoke-sensitive; MQ-5 is not.

Chapter 3: Obstacle Sensor

3.1 Definition

Definition: An obstacle sensor is a digital sensor used to detect whether an object is present.

It uses IR light, gives 1/0 output, and its range is adjusted by potentiometer.

3.2 Pins and interfacing

Pin	Connection/purpose
VCC	Arduino 5V
GND	Arduino GND
OUT	Digital output; Arduino digital input

Connect OUT to pin 2; read using digitalRead(2).

3.3 Working principle

1. IR emitter transmits IR.
2. Obstacle reflects IR back.
3. Receiver receives reflection.
4. Reflected amplitude above threshold → LOW (0).
5. No obstacle → no reflection → HIGH (1).

Condition	IR reflection	Output
Obstacle present	Reflected	LOW / 0
No obstacle	Not reflected	HIGH / 1

Memory: Obstacle → reflection → LOW.

3.4 Arduino code

```
int obstacle_pres = 0; void setup() { pinMode(2, INPUT); pinMode(13, OUTPUT); Serial.begin(9600); } void loop() { obstacle_pres = digitalRead(2); if (obstacle_pres == LOW) { Serial.print("Obstacle is present"); digitalWrite(13, HIGH); } else { digitalWrite(13, LOW); } delay(1000); }
```

Code part	Explanation
pinMode(2, INPUT);	Sensor input.
pinMode(13, OUTPUT);	LED output.
digitalRead(2);	Reads HIGH/LOW.
if LOW	Obstacle detected.
digitalWrite(13,HIGH)	LED ON.
delay(1000);	Wait 1 s.

Output: “Obstacle is present” and LED ON when obstacle detected.

3.5 Applications and exam points

- Obstacle-avoiding robots, automatic doors, parking assistance, object counting.
- Simple, low cost, adjustable range.
- IR emitter transmits; receiver receives reflection; obstacle=LOW.

Common confusion: obstacle present gives LOW, not HIGH.

Chapter 4: HC-SR04 Ultrasonic Distance Sensor

4.1 Definition

Definition: HC-SR04 is an ultrasonic distance sensor that measures object distance using ultrasonic sound waves.

It measures distance only; it cannot identify object type.

4.2 Construction and pins

It has a transmitter (sends pulses) and receiver (listens for reflected waves).

Pin	Purpose
VCC	Arduino 5V
GND	Arduino GND
Trigger	Sends ultrasonic pulse
Echo	Receives reflected echo

Typical: Trigger→11, Echo→12.

4.3 Features

Feature	Value
Input voltage	5 V
Max current	20 mA
HIGH output	5 V
LOW output	0 V
Temperature	-15°C to 70°C
Sensing angle	30° cone
Angle of effect	15° cone
Frequency	40 kHz
Sound speed	343 m/s
Range	2–400 cm

4.4 Working principle

1. Trigger HIGH for 10 µs.
2. Sensor sends 8 pulses at 40 kHz.
3. Pulses travel to object.
4. Echo becomes HIGH.
5. Object reflects pulses.
6. Echo becomes LOW when reflection received.
7. Arduino measures Echo HIGH duration.
8. Duration gives distance.

No reflection → Echo times out after 38 ms.

Reflected Echo pulse width: 150 µs to 25 ms.

Memory: Trigger sends, Echo listens, time gives distance.

4.5 Timing diagram

1. Trigger HIGH 10 µs.
2. Sound generated.
3. Echo HIGH.
4. Sound travels and returns.
5. Echo LOW on reception.
6. Echo HIGH duration = round-trip time.

Divide Echo time by 2: sound travels to object and back.

4.6 Arduino functions

Function	Purpose	Example
delayMicroseconds()	Pause in µs	delayMicroseconds(10);
pulseIn()	Measure pulse duration	duration=pulseIn(echoPin,HIGH);

4.7 Distance formula

```
Distance(m)=Speed(m/s)×Time(s)
343 m/s=0.0343 cm/µs
Distance(cm)=(Duration÷2)×0.0343
Distance(cm)=(Duration÷2)÷29.1
```

Example: duration=582 µs → one-way=291 µs → distance≈10 cm.

Memory: Divide by 2 because round trip.

4.8 Arduino code

```
int trigPin=11; int echoPin=12; long duration, cm, inches; void setup(){ Serial.begin(9600);
pinMode(trigPin,OUTPUT); pinMode(echoPin,INPUT); } void loop(){ digitalWrite(trigPin,LOW);
delayMicroseconds(5); digitalWrite(trigPin,HIGH); delayMicroseconds(10); digitalWrite(trigPin,LOW);
duration=pulseIn(echoPin,HIGH); cm=(duration/2)/29.1; inches=(duration/2)/74; Serial.print(inches);
Serial.print("in, "); Serial.print(cm); Serial.print("cm"); Serial.println(); delay(250); }
```

Code part	Explanation
trigPin=11, echoPin=12	Connections.
Trigger OUTPUT, Echo INPUT	Arduino sends trigger, receives echo.
Trigger HIGH 10 µs	Starts measurement.
pulseIn(echoPin,HIGH)	Measures Echo duration.
cm formula	Converts to cm.
delay(250)	Waits 250 ms.

Output: distance in inches and cm on Serial Monitor.

4.9 Applications and exam points

- Obstacle-avoiding robots, parking sensors, distance measurement, water-level systems.
- Non-contact, good range, simple interfacing.
- 40 kHz, 8 pulses, 10 µs trigger, 2–400 cm, 343 m/s.

Common confusion: divide duration by 2; measures distance only.

Chapter 5: LDR Light Sensor

5.1 Definition

Definition: LDR (Light Dependent Resistor) changes resistance with light intensity.

Used in automatic street lights, night lamps, darkness detectors.

5.2 Working

Works on Ohm's law.

1. Light falls on LDR.
2. More light → resistance decreases.
3. Less light → resistance increases.
4. Used in voltage divider.
5. Voltage changes with resistance.
6. Arduino reads voltage.

Light	Resistance	Effect
High	Low	More current
Dark	High	Less current

Memory: More light → less resistance.

5.3 Voltage divider and interfacing

1. 10 kΩ resistor: 5V to A0.
2. LDR: A0 to GND.
3. Voltage divider formed.
4. Arduino reads A0.

Condition	LDR resistance	A0 voltage
Dark	High	Near 0 V
Bright	Low	Near 5 V

Memory: dark → low A0 voltage; bright → high A0 voltage.

5.4 Arduino code

```
void setup(){ pinMode(A0,INPUT); Serial.begin(9600); } void loop(){ int light_intensity=analogRead(A0); if(light_intensity<400){Serial.println("Darker");} else if(light_intensity>550){Serial.println("Too Bright");} delay(1000); }
```

Code	Explanation
analogRead(A0)	Reads LDR voltage.
<400	Darker.
>550	Too Bright.
delay(1000)	Wait 1 s.

Output: "Darker" or "Too Bright".

5.5 Applications and exam points

- Automatic street lights, night lamps, light alarms.
- Low cost, small, simple analog.
- LDR resistance changes with light; voltage divider; analogRead.

Common confusion: in given circuit dark = high resistance = A0 near 0 V.

Chapter 6: Heartbeat Sensor

6.1 Definition

Definition: Heartbeat/pulse sensor measures heartbeat/heart rate.

Used in healthcare and wearables; inexpensive; generates square wave per pulse; averaging gives analog voltage.

6.2 Pins

Pin	Connection
VCC	5V
GND	GND
A0/Signal	Arduino A0

6.3 Working

1. Finger on sensor.
2. Light shines on finger.
3. Heartbeat → blood volume in capillaries increases.
4. Between beats volume decreases.
5. Light transmission/reflection changes.
6. Sensor converts to electrical pulse.
7. Arduino calculates BPM.

Flow: heartbeat → blood change → light change → pulse → Arduino → BPM.

6.4 BPM formula

Term	Meaning
BPM	Beats per minute
UpperThreshold	518
LowerThreshold	490
millis()	ms since start/reset
PulseInterval	Time between pulses (ms)

```
BPM=(1/PulseInterval)×60×1000

BPM=60,000/PulseInterval(ms)
```

Example: 800 ms → 75 BPM.

6.5 Code logic

```
reading=analogRead(A0); if(reading>UpperThreshold && !IgnoreReading){
if(!FirstPulseDetected){FirstPulseTime=millis();FirstPulseDetected=true;}
else{SecondPulseTime=millis();PulseInterval=Second-First;First=Second;} IgnoreReading=true; }
if(reading<LowerThreshold){IgnoreReading=false;} BPM=(1.0/PulseInterval)*60.0*1000;
```

Part	Explanation
reading>A	Pulse leading edge.
millis()	Stores time.
PulseInterval	Time between beats.
IgnoreReading	Prevents double count.
BPM formula	Calculates BPM.

Output: BPM on Serial Monitor.

6.6 Applications and exam points

- Fitness bands, health monitoring, wearables.
- Low cost, non-invasive.
- Blood-volume/light change; millis(); BPM from interval.

Common confusion: sensor gives pulses; Arduino calculates BPM.

Chapter 7: Actuators

7.1 Definition

Definition: Actuator moves or controls a mechanism/system according to control signal.

Sensors sense; controller decides; actuator acts. Examples: LED, motor, relay, servo, fan, pump.

7.2 Working

Flow: Sensor data → Controller → Control signal → Actuator → Action.

Example: gas sensor → Arduino → relay → fan/alarm ON.

7.3 Types

Type	Energy	Explanation
Hydraulic	Liquid pressure	Mechanical operation.
Pneumatic	Compressed air/vacuum	Linear or rotary motion.
Electrical	Electricity	Motion or switching.

Memory: Hydraulic=liquid, Pneumatic=air, Electrical=electricity.

7.4 Sensor vs actuator

Point	Sensor	Actuator
Function	Detects	Acts
Type	Input	Output
Connection	Input pins	Output pins/driver
Examples	LDR, MQ-2	Relay, servo

8. Relay

Definition: Relay is electrically activated switch; allows low-power Arduino to control high-power load.

5V relay module, 10A capacity; transistor switches relay; coil creates magnetic field.

8.1 Terminals

Terminal	Meaning
COM	Common
NC	Normally Closed; COM–NC when OFF
NO	Normally Open; COM–NO when ON

Memory: OFF→NC; ON→NO.

8.2 Working

1. OFF: no coil current; COM–NC.
2. Arduino sends signal.
3. Coil current flows.
4. Magnetic field moves switch.
5. COM disconnects NC, connects NO.
6. Load powered.

State	Coil	COM
OFF	No current	NC
ON	Current	NO

8.3 Connection and code

- Arduino 5V→relay VIN; GND→GND; D6→Signal.
- 230V AC example with bulb through relay contacts.

Safety: 230V AC dangerous; proper isolation needed.

```
int RelayPin=6; void setup(){pinMode(RelayPin,OUTPUT);} void
loop(){digitalWrite(RelayPin,HIGH);delay(3000);digitalWrite(RelayPin,LOW);delay(3000);}
```

Output: relay ON 3 s, OFF 3 s repeatedly.

8.4 Applications

- Switching circuits, home automation, heavy loads, safety circuits, automobile electronics.

Common confusion: Arduino cannot directly control AC load; relay is interface.

Chapter 9: Servo Motor

9.1 Definition

Servo: closed-loop control system using feedback. **Servo motor:** actuator moving to specified angular/linear position using position feedback.

Can rotate to 90°, 180°, etc. Arduino Servo library generates control signal.

9.2 Working

1. Arduino sends position command.
2. Servo moves.
3. Feedback measures actual position.
4. Compares with desired.
5. Moves until match.
6. Stops and holds.

Flow: Command → Movement → Feedback → Correction → Position.

9.3 Servo library

Controls many servos; 12 servos with 1 timer; Arduino Due up to 60. Earlier versions supported pins 9,10 only.

Method	Purpose	Syntax
attach()	Connect servo to pin	servo.attach(pin);
write()	Move to angle	servo.write(angle);
read()	Last written angle	servo.read();
attached()	Check connection	servo.attached();
detach()	Disconnect	servo.detach();

9.4 Code examples

```
#include <Servo.h> Servo myservo; void setup(){myservo.attach(9);}
myservo.write(90); // midpoint myservo.write(0); // start
int agl; agl=myservo.read();
bool joined; joined=myservo.attached();
myservo.detach(9);
```

9.5 Arduino code

```
#include <Servo.h> Servo darshanservo,bscitservo,bcaservo; void
setup(){Serial.begin(9600);bscitservo.attach(9);bcaservo.attach(10);darshanservo.attach(11);} void
loop(){darshanservo.write(10);bcaservo.write(45);bscitservo.write(110);delay(15);}
```

Code	Explanation
include Servo.h	Library.
Servo objects	Three servos.
attach(9/10/11)	Connect to pins.
write(10/45/110)	Move to angles.
delay(15)	Wait 15 ms.

Output: servos move to 10°,45°,110°.

9.6 Applications and exam points

- Robotic arms, door locks, camera pan-tilt, gates.
- Accurate position control, feedback-based.
- Closed-loop, position feedback, write(angle), attach(pin).

Common confusion: servo ≠ normal DC motor; servo holds position with feedback.

Chapter 10: Complete Final Revision

10.1 Summary

Topic	Key points
Sensors	Input; physical→electrical; active/passive; analog/digital.
MQ-02/05	Gas/smoke; gas reduces resistance; AO analog.
Obstacle	IR; obstacle=LOW; potentiometer range.
HC-SR04	Ultrasonic; Trigger/Echo; 2–400 cm; formula.
LDR	Light changes resistance; voltage divider.
Heartbeat	Blood/light change → pulse → BPM.
Actuators	Output; hydraulic/pneumatic/electrical.
Relay	Switch; OFF=NC, ON=NO.
Servo	Closed-loop position; attach/write/read/attached/detach.

10.2 Definitions

- Sensor: detects environment changes → electrical signal.
- Actuator: moves/controls mechanism.
- Analog: continuous output.
- Digital: HIGH/LOW.
- Active: needs power.
- Passive: no external power.
- Relay: electrically activated switch.
- Servo: closed-loop position actuator.
- BPM: beats per minute.

10.3 Formulas/syntax

Item	Formula/syntax	Use
Distance	cm=(duration/2)/29.1	HC-SR04
Distance alt	cm=(duration/2)×0.0343	Same
Speed	343 m/s=0.0343 cm/µs	Conversion
BPM	BPM=(1/PulseInterval)×60×1000	Heartbeat
BPM simple	BPM=60,000/PulseInterval(ms)	Interval in ms
analogRead	analogRead(A0)	Analog sensor
digitalRead	digitalRead(pin)	Digital sensor
digitalWrite	digitalWrite(pin,HIGH/LOW)	Output
servo.attach	servo.attach(pin);	Connect servo
servo.write	servo.write(angle);	Move servo

10.4 Comparisons

Concept	Difference
Analog vs digital	Continuous vs HIGH/LOW.
Active vs passive	Power vs no power.
Sensor vs actuator	Input vs output.
MQ-2 vs MQ-5	Smoke-sensitive vs not.
Trigger vs Echo	Send vs receive.
NC vs NO	OFF vs ON.
DC vs servo	Speed vs position feedback.

10.5 Checklist

- IoT flow.
- Classifications.
- MQ-2 gases/working.
- Obstacle IR.
- HC-SR04 details/formula.
- LDR principle.
- Heartbeat principle/BPM.
- Actuator types.
- Relay terminals.
- Servo methods.

Chapter 11: Exam Practice Questions

11.1 Short answers

Q1. Sensor: detects environment changes → electrical signal.

Q2. Actuator: moves/controls mechanism per control signal.

Q3. Analog continuous; digital HIGH/LOW.

Q4. MQ-2: smoke,H2,LPG,CH4,alcohol.

Q5. Obstacle present → LOW.

Q6. Trigger sends; Echo receives.

Q7. Divide by 2: round trip.

Q8. Light ↑ → LDR resistance ↓.

Q9. Blood-volume change → light change → pulse.

Q10. Relay: electrically activated switch.

Q11. NC when OFF; NO when ON.

Q12. Servo closed-loop: position feedback.

11.2 Long answers

Q1 MQ-02: definition, pins, gas reduces resistance, AO→A0, analogRead, applications.

Q2 HC-SR04: Trigger 10 µs → 8 pulses 40 kHz → Echo time → distance formula → code.

Q3 LDR: resistance changes with light; voltage divider; A0; dark/bright.

Q4 Heartbeat: blood-volume/light change → pulse → millis interval → BPM.

Q5 Relay: COM/NC/NO; OFF/ON; Arduino D6; applications.

Q6 Servo: closed-loop feedback; library methods; example 90°.

11.3 MCQs

1. Sensor = input — B.

2. MQ-2 — C.

3. Gas → resistance decreases — B.

4. Obstacle → LOW — B.

5. 8 pulses/40 kHz — B.

6. 10 µs — A.

7. 2–400 cm — A.

8. Light ↑ resistance ↓ — B.

9. pulseIn — C.

10. millis — B.

11. Relay ON COM–NO — B.

12. Servo closed-loop — B.

13. write — C.

14. Pneumatic — B.

15. HC-SR04 — C.

Final One-Page Revision

Core rule

Sensor detects → Arduino processes → Actuator performs.

Key facts

- Sensor=input; actuator=output.
- Analog=continuous; digital=HIGH/LOW.
- MQ-2 gas/smoke; gas↑→resistance↓.
- Obstacle: IR reflection; obstacle=LOW.
- HC-SR04: Trigger/Echo; distance formula.
- LDR: light↑→resistance↓.
- Heartbeat: blood/light→pulse→BPM.
- Relay: OFF=NC, ON=NO.
- Servo: closed-loop position.

Functions

- analogRead(A0)
- digitalRead(pin)
- digitalWrite(pin,HIGH/LOW)
- delay(ms)
- delayMicroseconds(µs)
- pulseIn(pin,HIGH/LOW)
- millis()
- servo.attach/write/read

Answer rule

Sensor: definition→pins→working→interfacing→code/formula→applications. Actuator: definition→type/terminals→working→connection/code→applications.